PATRONES
de Diseño

Mabel Tatiana Chaparro Sanchez
Paula Alejandra Mora Montaña
Ficha: 3315796

ÍNDICE
de contenidos

01 QUÉ ES Y PARA QUE SIRVE

02 CARACTERÍSTICAS

03 COMO FUNCIONA Y COMO SE ORGANIZA

04 DIAGRAMA REPRESENTACIÓN GRAFICA

05 VENTAJAS Y DESVENTAJAS

06 TECNOLOGIAS RELACIONADAS

ÍNDICE
de contenidos

07 DEMOSTRACIÓN CODIGO

08 CONCLUSIÓN

¿QUÉ ES Y PARA QUE SIRVE?

Abstract Factory

¿ QUE ES ?

Fábrica de fábricas que genera familias de objetos relacionados.

¿ PARA QUE SIRVE ?

Garantiza que los objetos creados sean compatibles entre sí sin acoplar el código a sus clases concretas.

Builder

¿ QUE ES ?

Constructor paso a paso para objetos complejos.

¿ PARA QUE SIRVE ?

Permite crear distintas configuraciones o representaciones de un objeto complejo usando la misma lógica de construcción.

Prototype

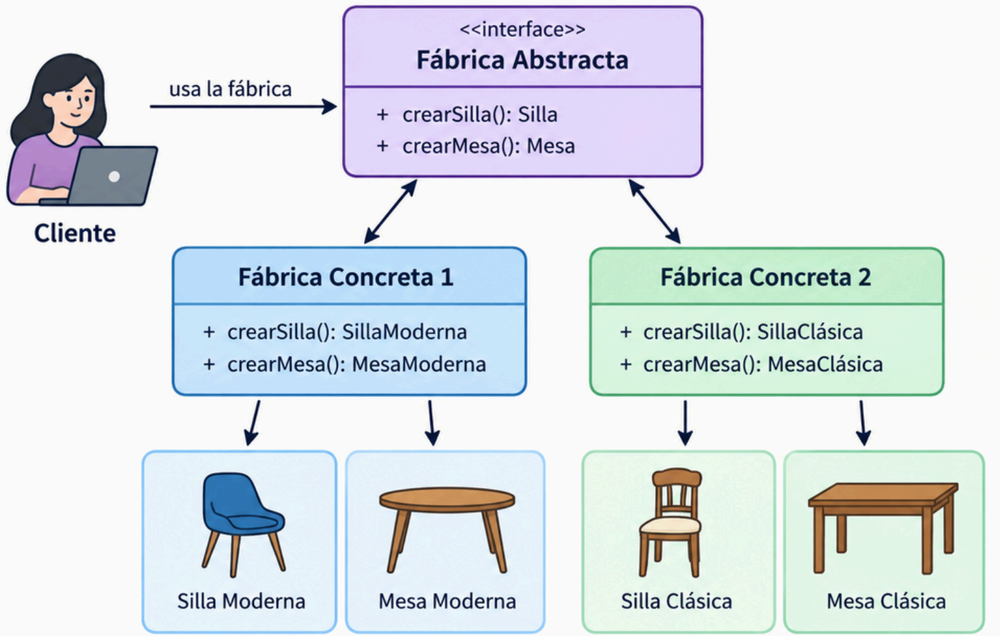
¿ QUE ES ?

Clonador de objetos existentes.

¿ PARA QUE SIRVE ?

Permite duplicar instancias sin depender de sus clases concretas y evitando el costo computacional de instanciar desde cero.

ABSTRACT FACTORY



- **Características:**

Trabaja con familias completas de productos; enfatiza la interfaz común.

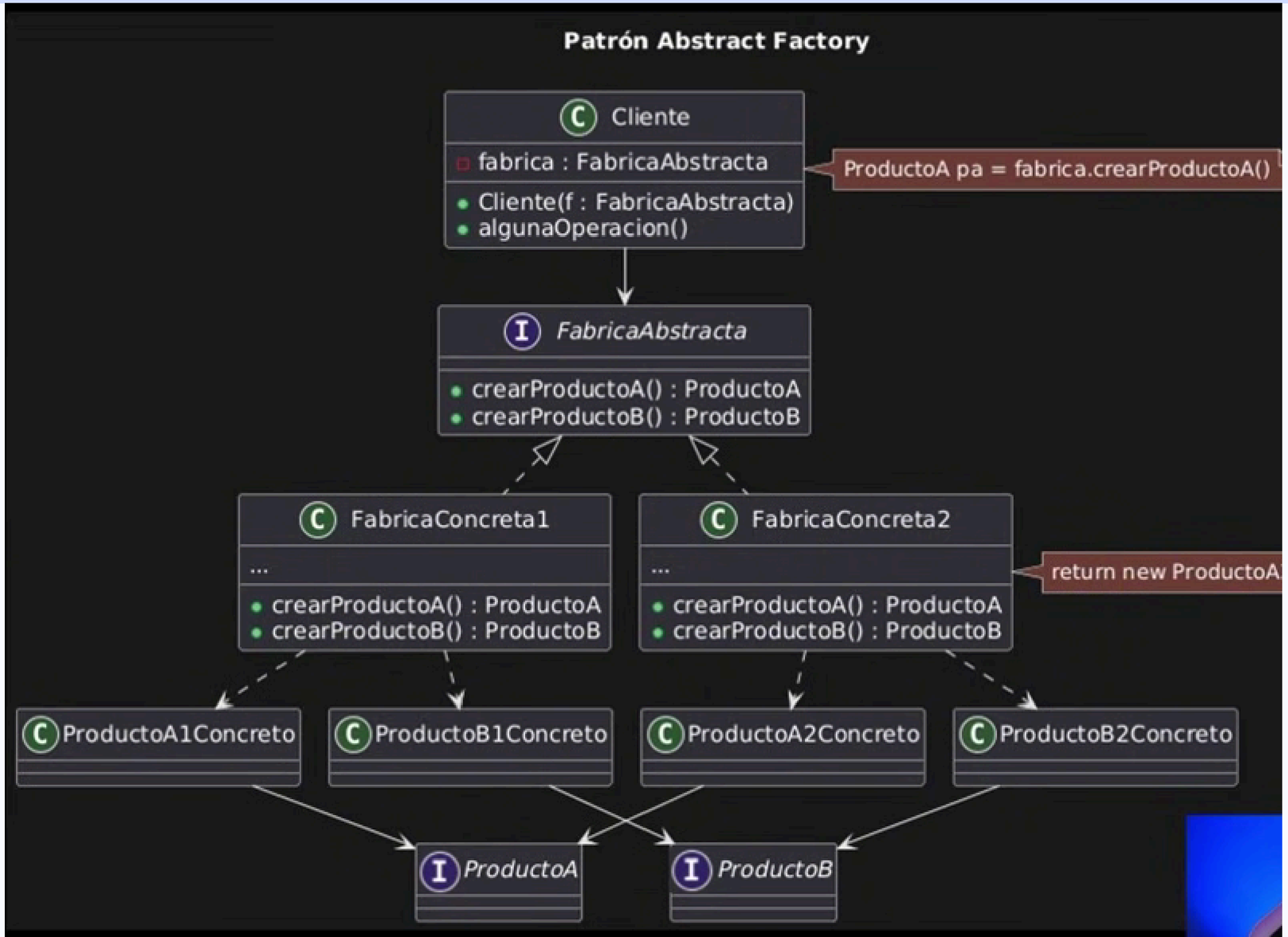
- **Funcionamiento:**

El cliente solicita una familia de productos a una interfaz abstracta; la fábrica concreta determina qué clases concretas instanciar.

- **Organización:**

AbstractFactory: Interfaz con métodos para crear cada producto.
ConcreteFactory: Implementa la creación de productos de una familia específica.
AbstractProduct: Interfaz para un tipo de producto.
ConcreteProduct: Implementación específica del producto.

I.DIAGRAMA DE REPRESENTACIÓN GRAFICA Y SUS CONEXIONES



VENTAJAS

- Garantiza la compatibilidad entre los productos de una familia.
- Reduce el acoplamiento entre el cliente y las clases concretas.
- Centraliza la creación de objetos.
- Facilita agregar nuevas variantes de productos.

DESVENTAJAS

- Introduce más interfaces y clases.
- Aumenta la complejidad inicial.
- Agregar un nuevo tipo de producto puede obligar a modificar todas las fábricas concretas.

```java
// 1. Interfaces de los productos
interface Boton {
    void dibujar();
}

interface Checkbox {
    void dibujar();
}

// 2. Fábrica abstracta
interface Fabrica {
    Boton crearBoton();
    Checkbox crearCheckbox();
}
```

```java
// 3. Productos concretos: familia Windows
class BotonWindows implements Boton {
    public void dibujar() {
        System.out.println("Botón de Windows");
    }
}

class CheckboxWindows implements Checkbox {
    public void dibujar() {
        System.out.println("Checkbox de Windows");
    }
}
```

```java
// 3. Productos concretos: familia Mac
class BotonMac implements Boton {
    public void dibujar() {
        System.out.println("Botón de Mac");
    }
}

class CheckboxMac implements Checkbox {
    public void dibujar() {
        System.out.println("Checkbox de Mac");
    }
}
```

```java
// 4. Fábricas concretas
class FabricaWindows implements Fabrica {
    public Boton crearBoton() {
        return new BotonWindows();
    }
    public Checkbox crearCheckbox() {
        return new CheckboxWindows();
    }
}

class FabricaMac implements Fabrica {
    public Boton crearBoton() {
        return new BotonMac();
    }
    public Checkbox crearCheckbox() {
        return new CheckboxMac();
    }
}
```

```java
// 5. Clase principal (cliente)
public class Main {
    public static void main(String[] args) {

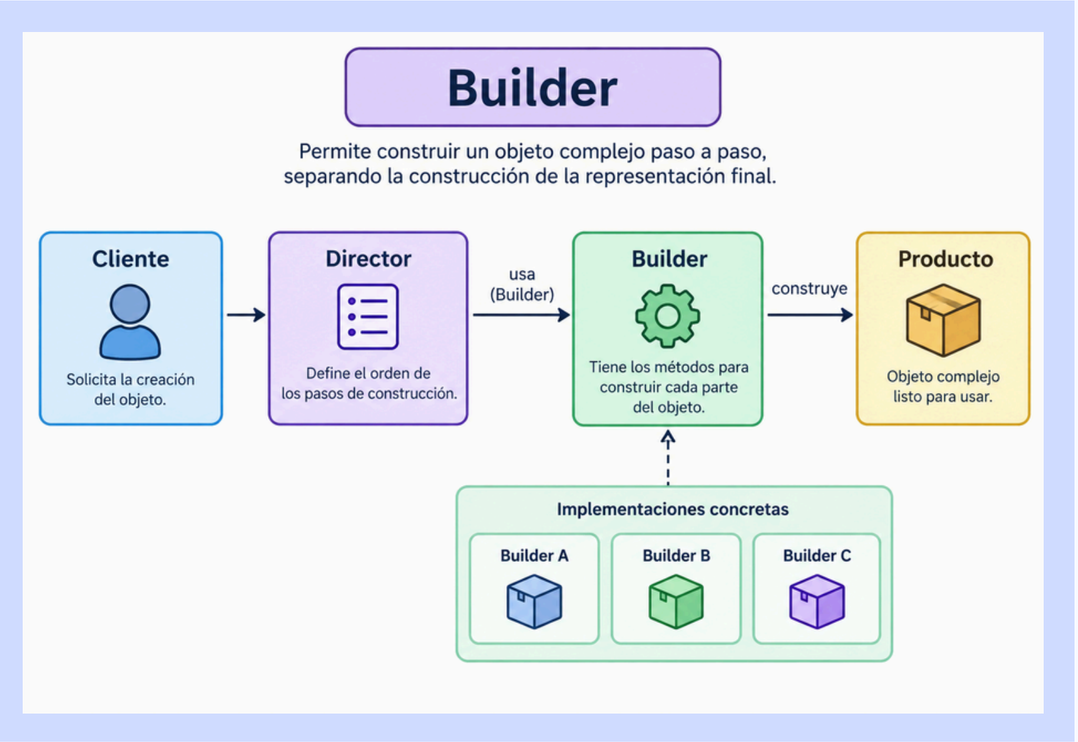
        // Familia Windows
        Fabrica fabrica = new FabricaWindows();
        Boton boton = fabrica.crearBoton();
        Checkbox checkbox = fabrica.crearCheckbox();

        boton.dibujar();
        checkbox.dibujar();

        // Para cambiar de familia solo se cambia la fábrica
        fabrica = new FabricaMac();
        boton = fabrica.crearBoton();
        checkbox = fabrica.crearCheckbox();

        boton.dibujar();
        checkbox.dibujar();
    }
}
```

BUILDER



- **Características:**

Construcción paso a paso, encadenamiento de métodos (method chaining) y separación de la lógica de ensamble respecto a la estructura final.

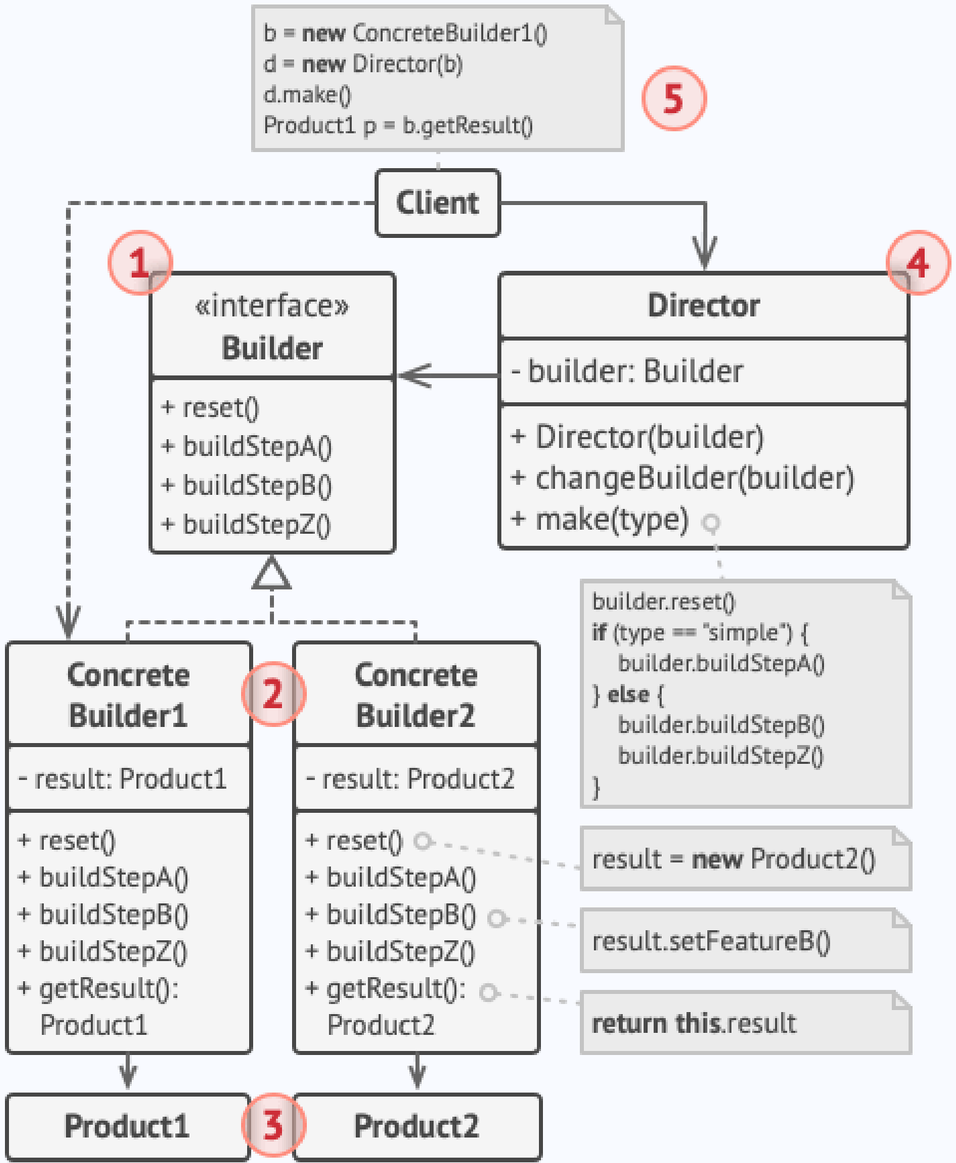
- **Funcionamiento:**

El cliente invoca de forma secuencial métodos de configuración en el constructor y al final invoca un método de entrega (build()).

- **Organización:**

Builder: Interfaz que declara los pasos de construcción.
ConcreteBuilder: Implementa los pasos y almacena el resultado final.
Director (opcional): Define el orden específico para ejecutar los pasos de construcción.
Product: Objeto complejo resultante.

2.DIAGRAMA DE REPRESENTACIÓN GRAFICA Y SUS CONEXIONES



VENTAJAS

- Evita constructores demasiado largos.
- Facilita construir objetos complejos paso a paso.
- Permite reutilizar secuencias de construcción.
- Hace más clara la configuración de los objetos.

DESVENTAJAS

- Añade clases o métodos que pueden aumentar el código.
- Puede ser excesivo para objetos sencillos.
- Requiere diseñar y mantener los pasos de construcción.

```java
//1 Objeto a construir
public class Casa {
    private String tipoEstructura;
    private int pisos;
    private boolean piscina;
    private boolean garage;
    private boolean jardin;
    private List<String> extras = new ArrayList<>();


    //setters (para que el builder pueda armar la casa paso a paso)
    public void setTipoEstructura(String tipoEstructura) {
        this.tipoEstructura = tipoEstructura;
    }

    public void setPisos(int pisos) {
        this.pisos = pisos;
    }

    public void setPiscina(boolean piscina) {
        this.piscina = piscina;
    }

    public void setGarage(boolean garage) {
        this.garage = garage;
    }

    public void setJardin(boolean jardin) {
        this.jardin = jardin;
    }
```

```java
public void setExtras(List<String> extras) {
    this.extras = extras;
}

@Override
public String toString() {
    return "Casa{" +
            "tipoEstructura='" + tipoEstructura + '\'' +
            ", pisos=" + pisos +
            ", piscina=" + piscina +
            ", garage=" + garage +
            ", jardin=" + jardin +
            ", extras=" + extras +
            '}';
}
```

```java
//2 Builder General
public interface BuilderCasa {

    void reiniciar();
    void construirEstructura(String estructura);
    void contruirPisos(int cantidad);
    void construirPiscina(boolean siONo);
    void construirGarage(boolean siONo);
    void construirJardin(boolean siONo);
    void agregarExtra(String extra);

}
```

Paso 3: ->

```java
import java.util.ArrayList;
import java.util.List;

//3 Builder Concreto
public class BuilderConcretoCasa implements BuilderCasa{

    private Casa resultado;
    private List<String> extras = new ArrayList<>();

    @Override
    public void reiniciar() {
        this.resultado = new Casa();
        this.extras = new ArrayList<>();
    }

    @Override
    public void construirEstructura(String tipoEstructura) {
        resultado.setTipoEstructura(tipoEstructura);
    }

    @Override
    public void contruirPisos(int cantidad) {
        resultado.setPisos(cantidad);
    }

    @Override
    public void construirPiscina(boolean siONo) {
        resultado.setPiscina(siONo);
    }
```

```java
    @Override
    public void construirGarage(boolean siONo) {
        resultado.setGarage(siONo);
    }

    @Override
    public void construirJardin(boolean siONo) {
        resultado.setJardin(siONo);
    }

    @Override
    public void agregarExtra(String extra) {
        this.extras.add(extra);
        resultado.setExtras(extras);
    }

    //Método típico de un Builder Concreto
    public Casa obtenerResultado() {
        return resultado;
    }
}
```

Director ->

```java
public class BuilderDirector {

    private BuilderCasa builder;

    //constructor propio de la clase
    public BuilderDirector(BuilderCasa builder) {
        this.builder = builder;
    }

    //para cambiar de constructor
    public void cambiarBuilder(BuilderCasa builder) {
        this.builder = builder;
    }

    //Casa de material, 2 pisos, con garage (sin piscina con jardín)
    public void construirCasaDosPisosMaterial() {
        builder.reiniciar();
        builder.construirEstructura("Material");
        builder.contruirPisos(2);
        builder.construirPiscina(false);
        builder.construirGarage(true);
        builder.construirJardin(true);
        builder.agregarExtra("Balcón");
    }
```

```java
    // Casa de material, 1 piso, con piscina, jardín y garage
    public void construirCasaConPiscina() {
        builder.reiniciar();
        builder.construirEstructura("Material");
        builder.contruirPisos(1);
        builder.construirPiscina(true);
        builder.construirGarage(true);
        builder.construirJardin(true);
        builder.agregarExtra("Deck alrededor de piscina");
    }

    // Casa madera simple con Jardín
    public void construirCasaMaderaSimple() {
        builder.reiniciar();
        builder.construirEstructura("Madera");
        builder.contruirPisos(1);
        builder.construirPiscina(false);
        builder.construirGarage(false);
        builder.construirJardin(true);
    }

}
```

```java
public static void main(String[] args) {

    //Builder Concreto
    BuilderConcretoCasa builder = new BuilderConcretoCasa();

    //Director (opcional)
    BuilderDirector director = new BuilderDirector(builder);

    //Creando los objetos
    //Casa de material, 2 pisos, con garage (sin piscina con jardín)
    director.construirCasaDosPisosMaterial();
    Casa casa1 = builder.obtenerResultado();
    System.out.println("Casa 1: " + casa1.toString());

    // Casa de material, 1 piso, con piscina, jardín y garage
    director.construirCasaConPiscina();
    Casa casa2 = builder.obtenerResultado();
    System.out.println("Casa 2: " + casa2.toString());

    // Casa madera simple con Jardín
    director.construirCasaMaderaSimple();
    Casa casa3 = builder.obtenerResultado();
    System.out.println("Casa 3: " + casa3.toString());

    //------------------------------------------------
    //El caso de no usar director
    builder.reiniciar();
    builder.construirEstructura("Material");
    builder.contruirPisos(5);
    builder.agregarExtra("Parrilla grande");
```

PROTOTYPE

- **Características:**

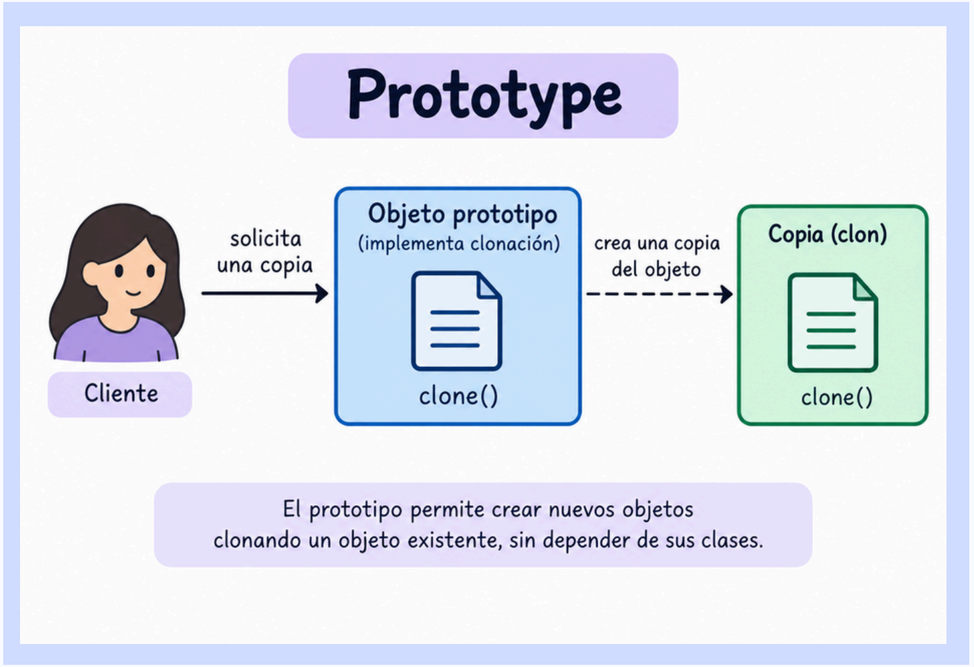
Utiliza la clonación en memoria (clone()), evita llamadas directas al operador new en la capa cliente.

- **Funcionamiento:**

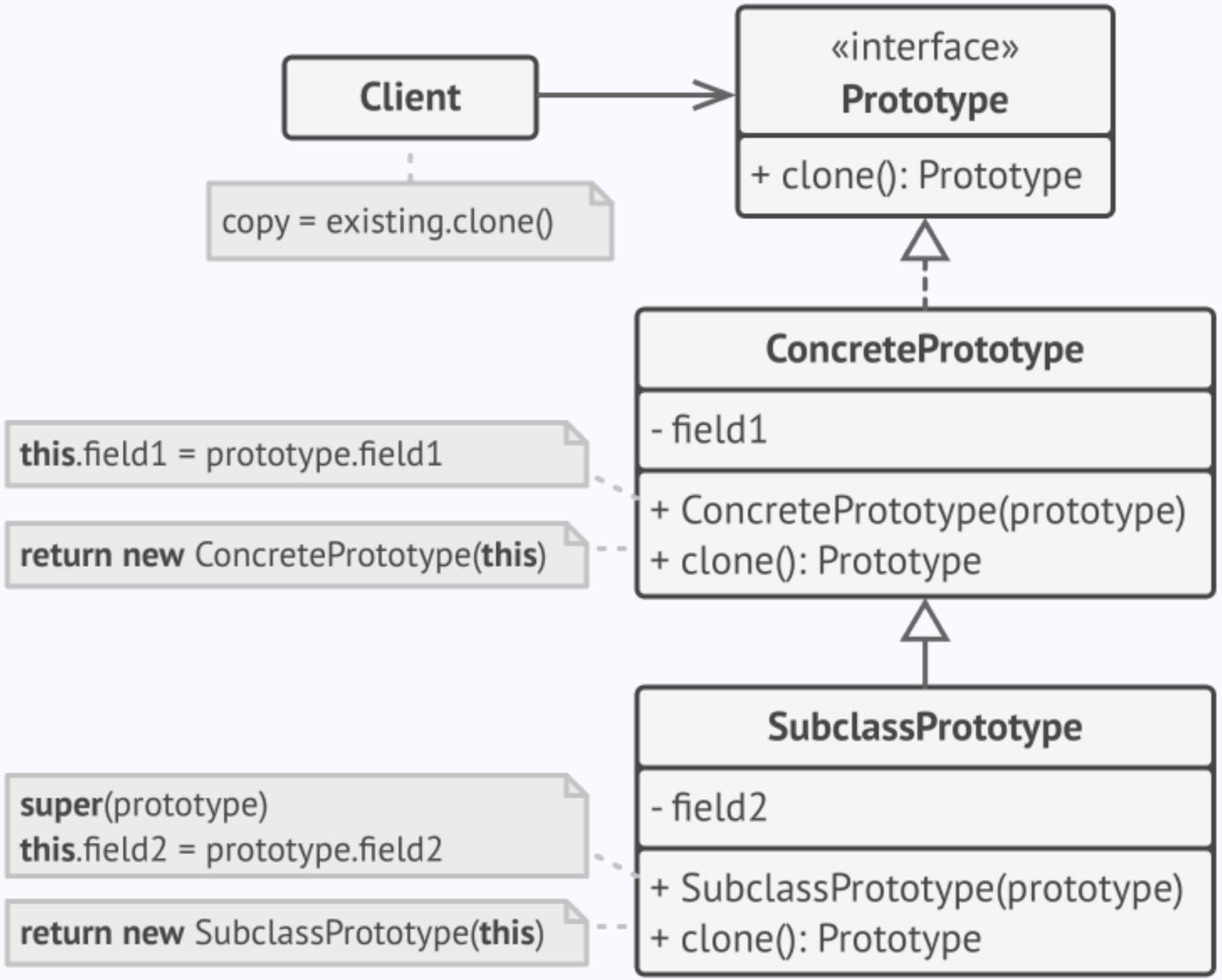
Un objeto existente declara un método para autoreproducirse haciendo una copia exacta de sus atributos en un nuevo espacio de memoria.

- **Organización:**

Prototype: Interfaz que declara el método clone().
ConcretePrototype: Implementa la lógica de copia (superficial o profunda).
Client: Solicita la clonación al prototipo.



3.DIAGRAMA DE REPRESENTACIÓN GRAFICA Y SUS CONEXIONES



VENTAJAS

- Permite reutilizar configuraciones existentes.
- Puede reducir el costo de crear objetos complejos.
- Facilita generar variantes de un mismo objeto.
- Evita repetir procesos de inicialización.

DESVENTAJAS

- Copiar objetos complejos puede ser difícil.
- Es necesario controlar las referencias a otros objetos.
- Una copia superficial puede compartir atributos internos con el original.
- Puede resultar innecesario cuando los objetos son sencillos.

```java

    //1 Creación del Prototipo
    public interface PrototipoPizza<T> {

        T clonar ();

    }
```

```java
import java.util.ArrayList;
import java.util.List;

//2 - El prototipo Concreto
public class PedidoPizza implements PrototipoPizza<PedidoPizza>{

    private String tamanio;
    private String tipoMasa;
    private List<String> ingredientes;

    //Constructor normal (crear desde cero)
    public PedidoPizza(String tamanio, String tipoMasa, List<String> ingredientes) {
        this.tamanio = tamanio;
        this.tipoMasa = tipoMasa;
        this.ingredientes = new ArrayList<>(ingredientes); //mutable
    }

    //Constructor copia
    public PedidoPizza(PedidoPizza prototipo) {
        this.tamanio = prototipo.tamanio;
        this.tipoMasa = prototipo.tipoMasa;
        this.ingredientes = new ArrayList<>(prototipo.ingredientes);
    }

    @Override
    public PedidoPizza clonar() {
        return new PedidoPizza(this);
    }
```

```java
    //un método para agregar los ingredientes
    public void agregarIngrediente (String ingrediente) {
       ingredientes.add(ingrediente);
    }

    @Override
    public String toString() {
        return "PedidoPizza{" +
                "tamanio='" + tamanio + '\'' +
                ", tipoMasa='" + tipoMasa + '\'' +
                ", ingredientes=" + ingredientes +
                '}';
    }
}
```

Paso 3: SubclassPrototype (opcional) ->

```java
import java.util.List;

public class PedidoPizzaEspecial extends PedidoPizza{

    private boolean bordeRelleno;


    public PedidoPizzaEspecial(String tamanio, String tipoMasa,
                               List<String> ingredientes, boolean bordeRelleno) {
        super(tamanio, tipoMasa, ingredientes);
        this.bordeRelleno = bordeRelleno;
    }

    public PedidoPizzaEspecial(PedidoPizzaEspecial prototipo) {
        super(prototipo);
        this.bordeRelleno = bordeRelleno;
    }

    @Override
    public PedidoPizzaEspecial clonar() {
        return new PedidoPizzaEspecial(this);
    }

    @Override
    public String toString() {
        return "PedidoPizzaEspecial{" +
                "bordeRelleno=" + bordeRelleno +
                " , " + super.toString() +
                '}';
    }
```

TECNOLOGÍAS RELACIONADAS

- Frameworks GUI (Java Swing, React, Flutter): Usan Abstract Factory para renderizar componentes visuales ajustados al sistema operativo o tema (Light/Dark mode).

- Bases de datos y Mapeadores ORM (Hibernate, Entity Framework, SQL Query Builders): Usan Builder para componer consultas SQL complejas de forma dinámica mediante sintaxis fluida (Select().Where().Join()).

- Sistemas de Caché y Motors de Rendimiento (Redes, Videojuegos): Usan Prototype para instanciar objetos pesados en memoria (ej. duplicar entidades o enemigos preconfigurados en un mapa).

CONCLUSIÓN

- Arquitectura Monolítica: El proyecto es pequeño o mediano, el equipo es reducido, se quiere sacar el sistema rápido (MVP) o el dominio aún no está claro.
- Abstract Factory: Hay que crear familias de objetos que deben ser compatibles entre sí y se quiere poder cambiar de familia fácilmente.
- Builder: El objeto es complejo, tiene muchos parámetros u opciones, o se arma por pasos.
- Prototype: Crear el objeto desde cero es costoso o se necesitan muchas copias parecidas de una plantilla.

MUCHAS GRACIAS